NAME: DAWOOD ASHRAF

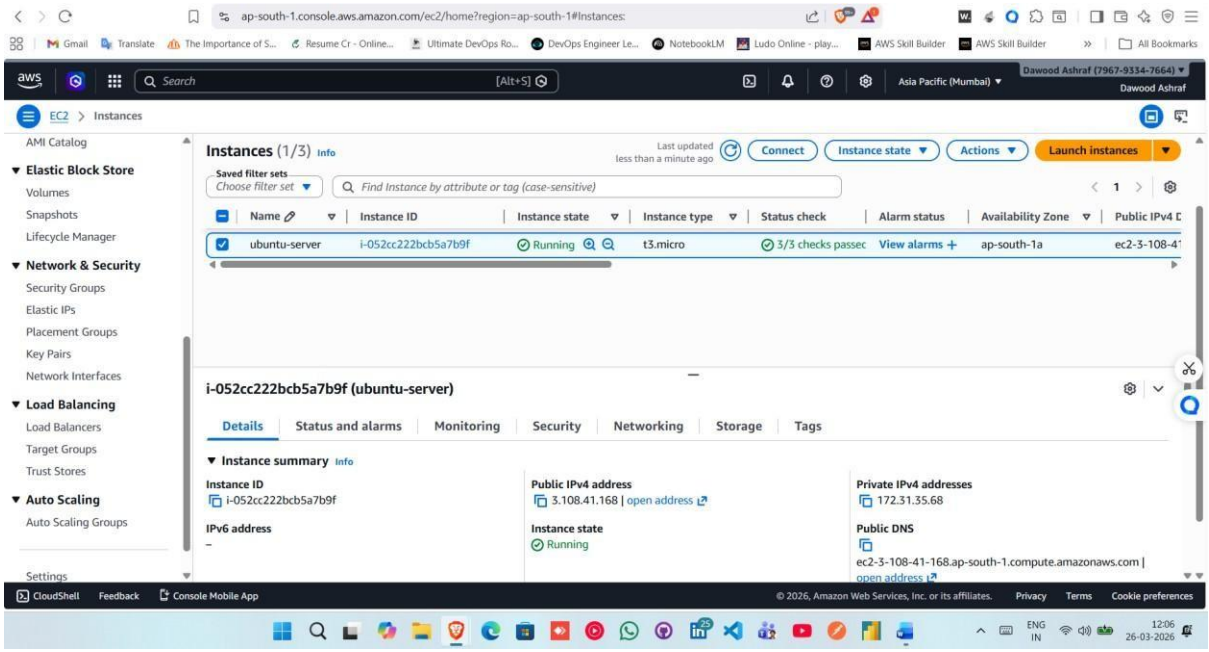
SUBJECT: VPN_PROJECT

DATE: 27-03-2026

STEP 1:

Create an ubuntu instance which will act as our server.

This server should route all traffic anywhere on internet.



STEP 2:

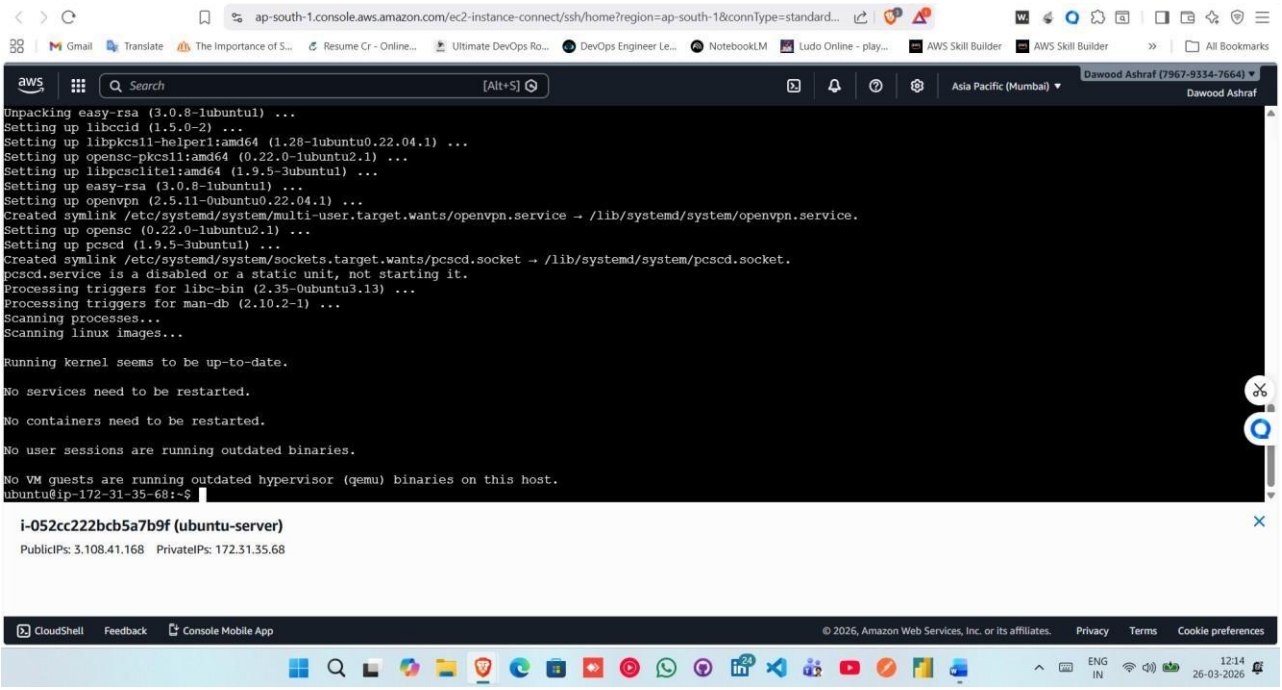
Connect to the EC2 instance and install openvpn and easy-rsa:

These are two core tools needed to create your own private VPN. In AWS context it is like creating a ***tunnel*** *between the local computer and the private EC2 instances in a VPC.*

OpenVPN: *This is the application that handles the actual encrypted connection. It creates the "tunnel" that allows data to travel securely over the public internet.*

Easy-RSA: *This is a Command Line Interface (CLI) utility used to manage* ***SSL Certificates****. Since OpenVPN uses certificates to verify who is allowed to connect, you need* ***easy-rsa*** *to act as your "Certificate Authority" (the digital entity that signs and issues the "ID cards" for your server and your laptop).*

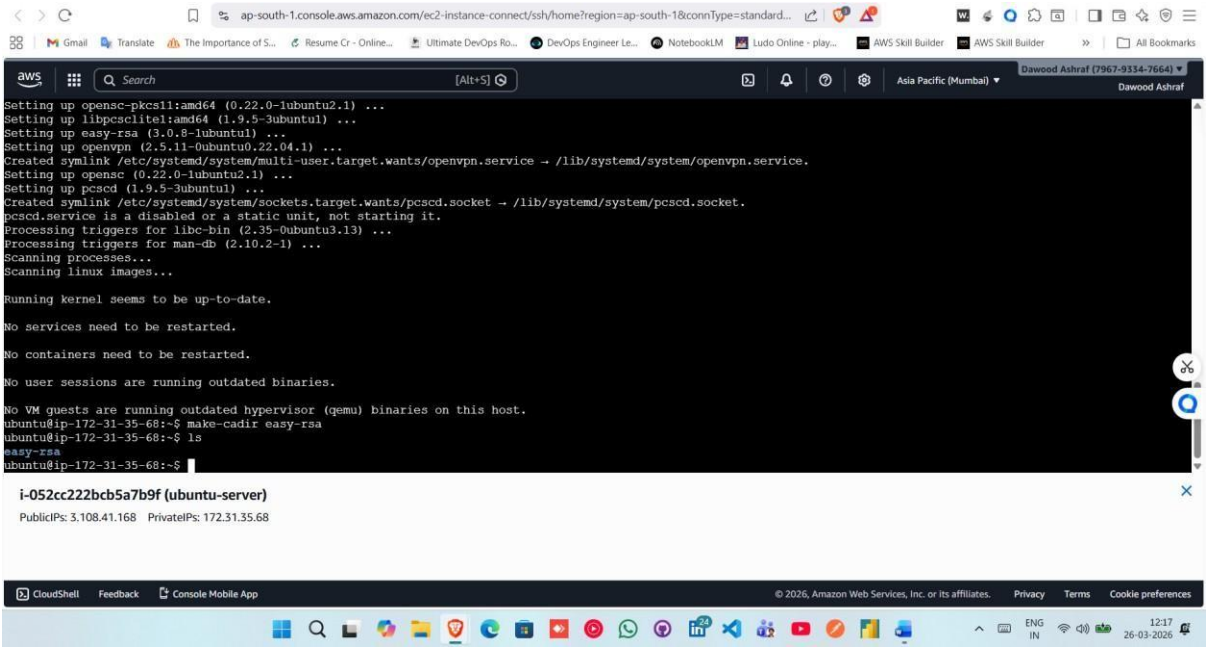
Command: ***<sudo apt-get install -y openvpn easy-rsa>***



STEP 3:

Make Certificate Authority Directory, you are telling the system to create a dedicated workspace where you can safely generate your security keys and certificates without messing up the original program files.

*Command: <**make-cadir easy-rsa**>*

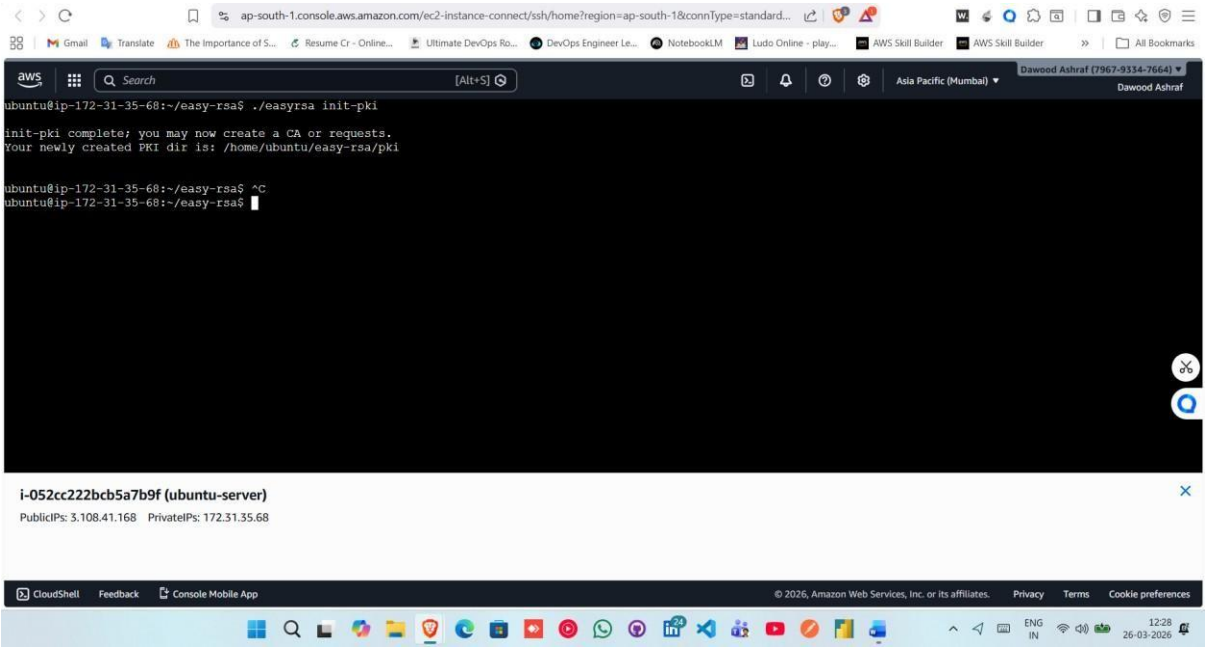


STEP 4:

Initialize PKI directory inside your easyrsa directory, where you can save all the future keys, requests, and certificates.

*Commqnd: <**./easyrsa init-pki**>*

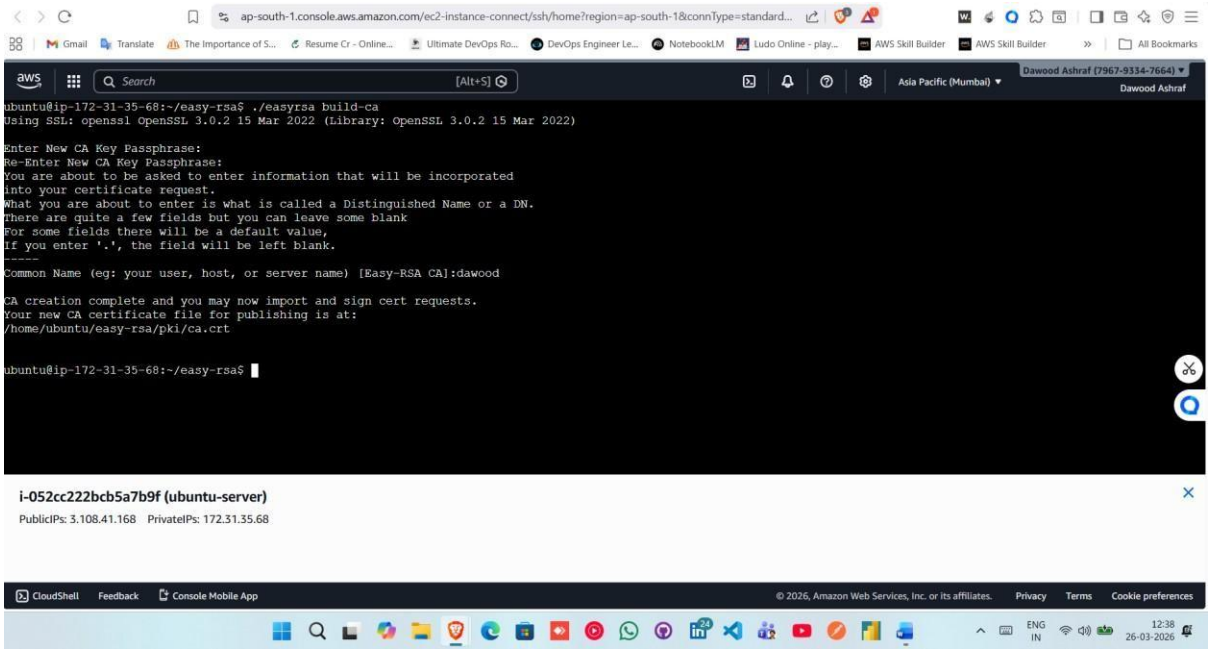
"./" means look into this one and run the script.



STEP 5:

*Create your **Certificate Authority (CA)**—the "Master Identity" that will sign and validate every other certificate in your network.*

*Command: **<./easyrsa build-ca>***



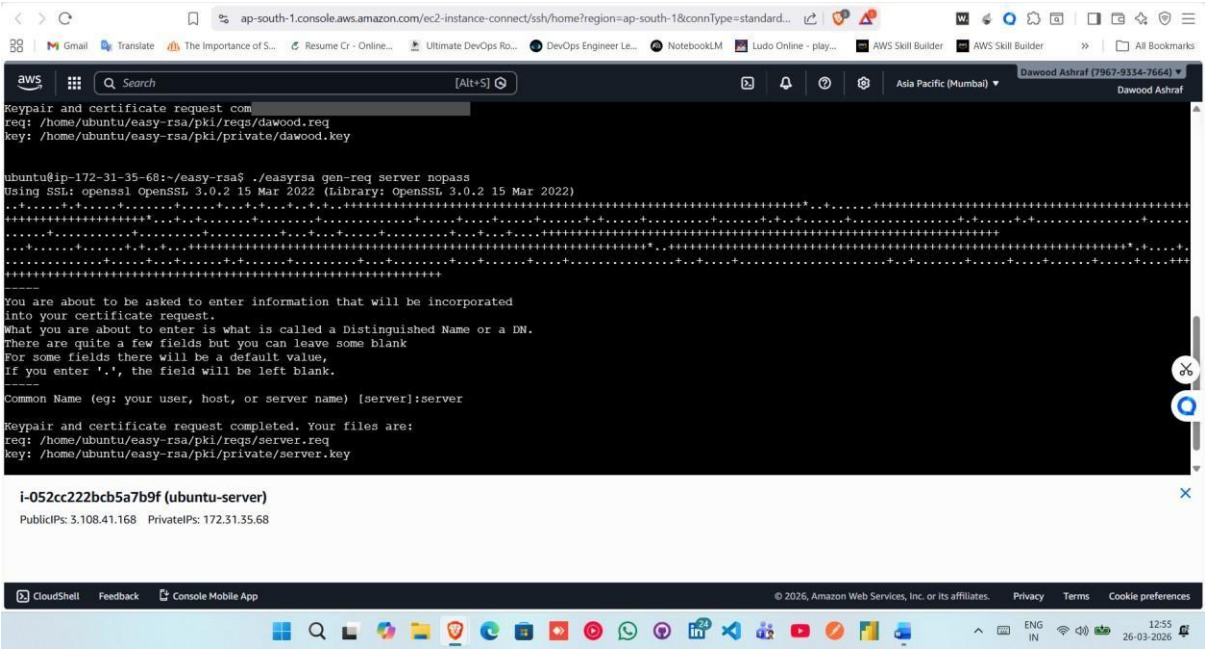
STEP 6:

Create an identity for your Server and Client which you need to sign in after generating the Certificate Signing Request ***(CSR) or*** *simply Request.*

Command: ***<./easyrsa gen-req server nopass>*** *for the server*

<./easyrsa gen-req user nopass> *for the client/user*

This will create a pair ***Signing Request*** *and a* ***Private key*** *for both, which will be later used for access.*

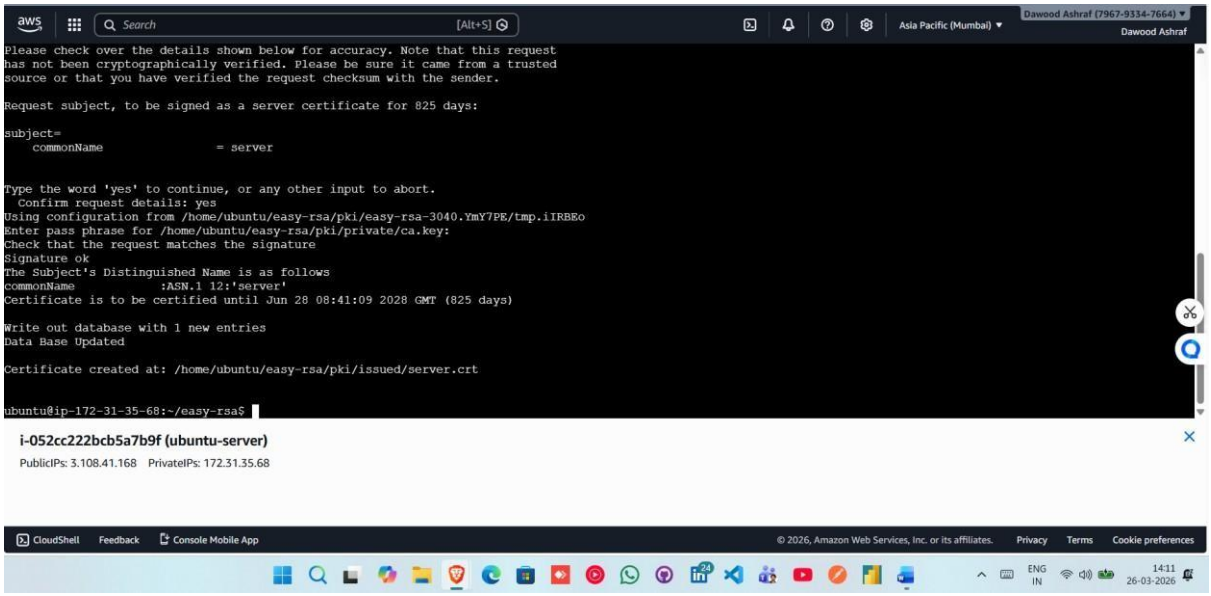


STEP 7

:

*Telling **certificate authority (CA)** to sign a pending request: Command:*

*<./**./easyrsa sign-req server server**>*



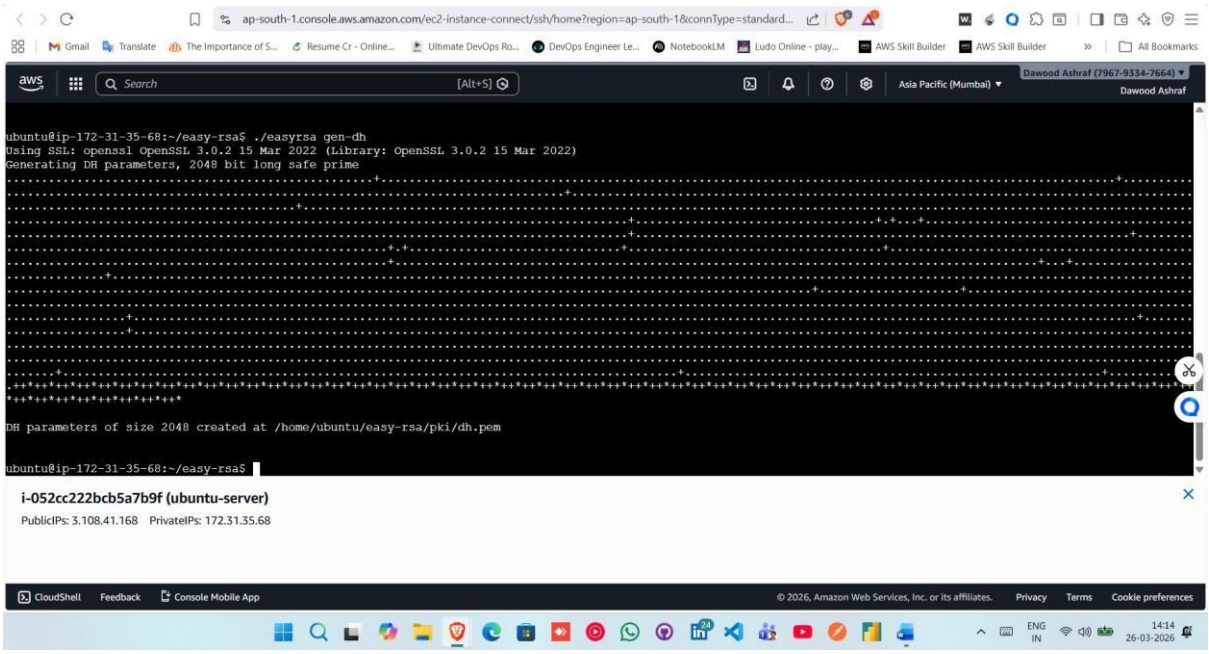
STEP 8

:

Final handshake math, used for initial connection.

Diffie-Hellman *is a mathematical algorithm that allows two parties (your VPN server and your laptop) to create a shared secret key over an untrusted network (the internet) without actually sending the key itself.*

Command: ***<./easyrsa gen-dh>***

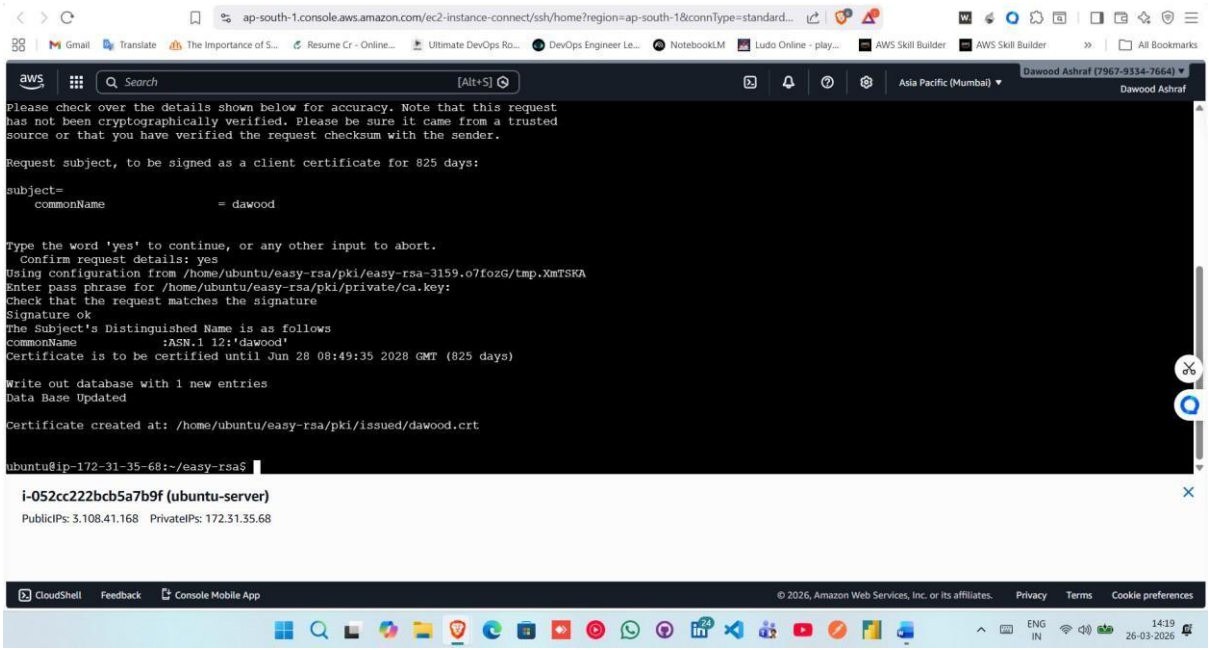


STEP 9

:

Create clients or users personal ID card to access the network.

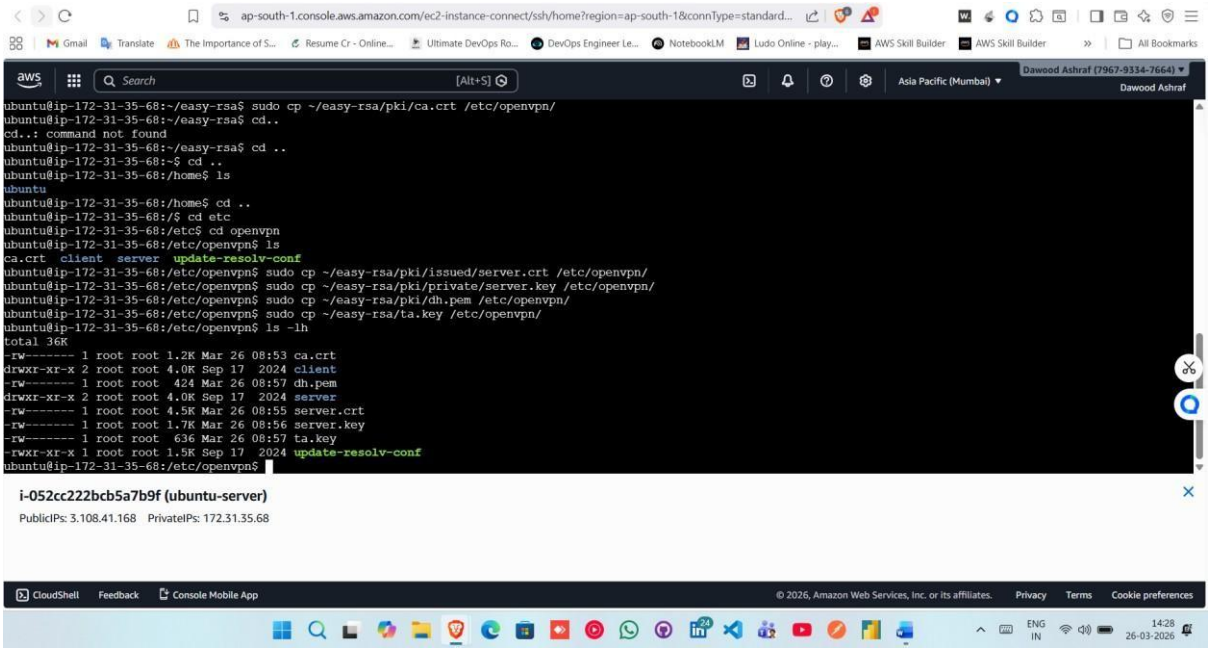
*Command: <./**easyrsa sign-req client dawood**>*



STEP 10

STEP 11:

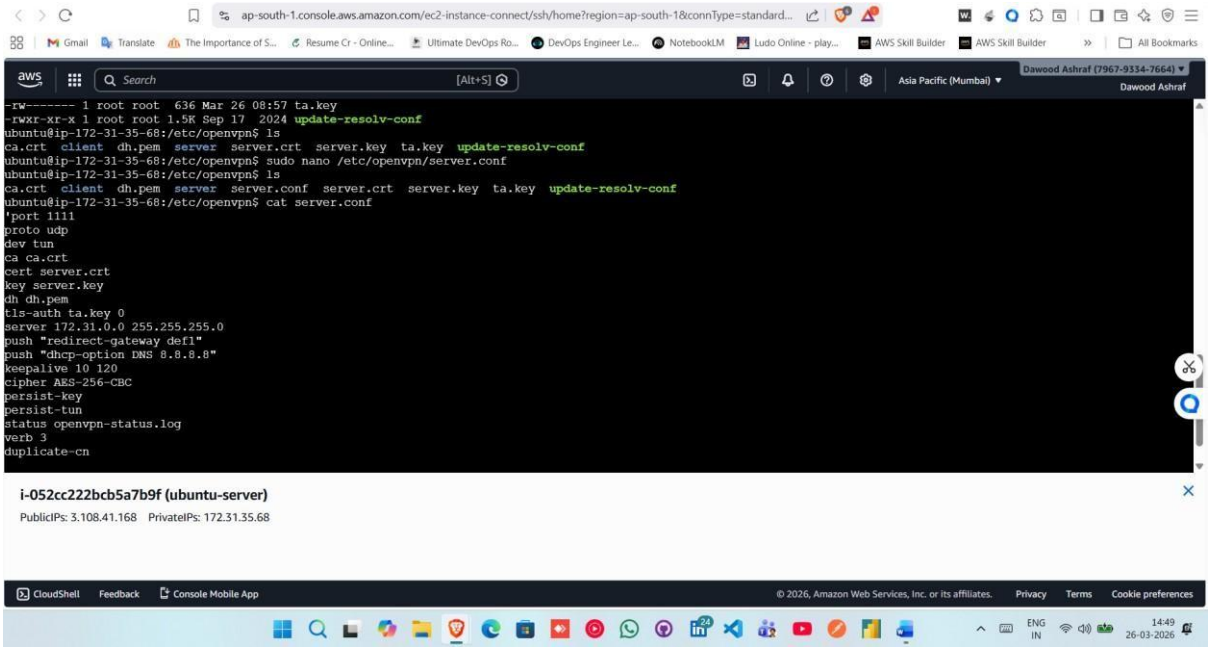
Copy all the required certificates and keys inside the openvpn directory inside etc directory of the root user.



STEP 12:

Create a server.conf file inside etc/openvpn

*The server.conf file is the **Main Instruction Manual** for the OpenVPN service on your EC2 instance. It tells the server exactly how to behave, which security keys to use, and how to treat incoming connections from clients like your laptop.*



STEP 13:

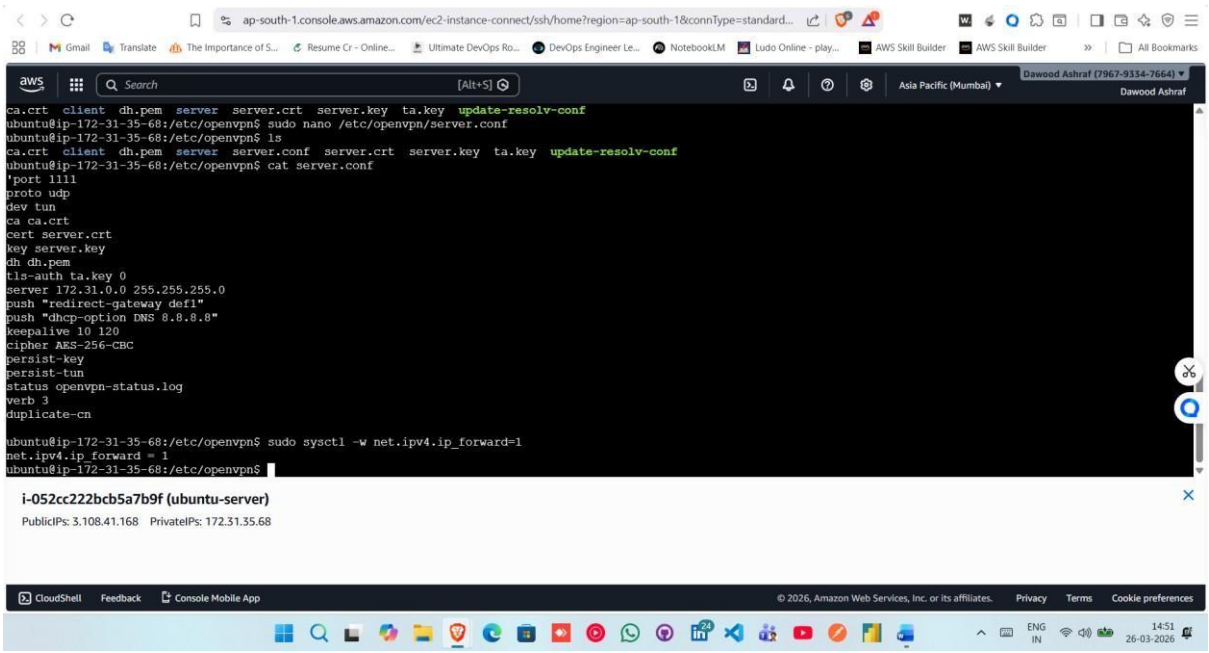
By default, Linux is security-conscious. If a packet arrives at your EC2 instance that isn't addressed to the instance itself, Linux will simply drop it.

*When you set **ip_forward=1**, you are telling the Linux Kernel: "If you receive data from my laptop (the VPN client) that is meant for the internet (like https://www.google.com/search?q=google.com), don't drop it. Instead, pass it through to the next hop."*

*Command : **<./sudo sysctl -w net.ipv4.ip_forward=1>***

The Breakdown

- ***sudo**: Runs the command with administrative (root) privileges.*
- ***sysctl**: The utility used to modify Linux kernel parameters at runtime.*
- ***-w:** Stands for "write." It applies the change immediately.*
- ***net.ipv4.ip_forward=1:** The specific setting. 0 is Off, 1 is On.*



--to make the ip_forward = 1 permanent we use below command as a root user:

STEP 14:

Command: <./ ***sudo echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf>***

And then check if it got permanent by using the following command:

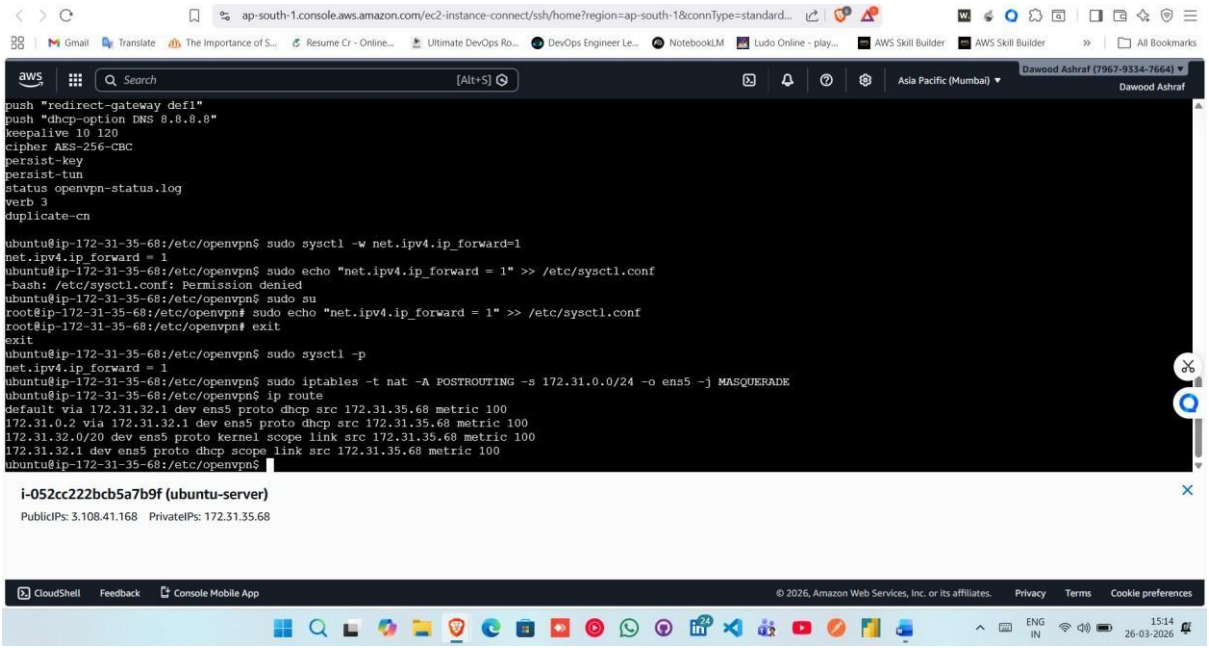
<./ ***sudo sysctl -p>***

the final "Golden Rule" that turns your EC2 instance into a functioning gateway. Without this command, your VPN would be like a phone line that connects but has no sound—you'd be "on" the VPN, but you wouldn't be able to load a single website.

The Breakdown: What this command actually does

- ***sudo iptables****: Invokes the administration tool for IPv4 packet filtering and NAT.*
- ***-t nat****: Tells the system to look at the* ***Network Address Translation*** *table. This is the specific "ledger" that handles IP address swapping.*
- ***-A POSTROUTING****: "Append" this rule to the* ***Postrouting*** *chain. This means "apply this rule to the packet just before it leaves the EC2 instance to go to the internet."*
- ***-s 172.31.0.0/24****: The* ***Source****. This tells the firewall to only apply this rule to traffic coming from your VPN network (the 172.31.x.x range we set in server.conf).*
- ***-o ens5****: The* ***Output Interface****. This is the name of your EC2 instance's actual internetconnected network card. (Note: On some AWS instances, this might be eth0. You can check by typing ip route).*
- ***-j MASQUERADE****: The* ***Action****. This is the magic part. It tells the EC2 instance: "Hide the private VPN IP of Dawood's laptop and replace it with my own Public AWS IP address so the internet can talk back to him."*

STEP 15:


ap-south-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?region=ap-south-1&connType=standard...
Asia Pacific (Mumbai)
Dawood Ashraf (7967-9334-7664)
Dawood Ashraf
push "redirect-gateway def1"
push "dhcp-option DNS 8.8.8.8"
keepalive 10 120
cipher AES-256-CBC
persist-key
persist-tun
status openvpn-status.log
verb 3
duplicate-cn
ubuntu@ip-172-31-35-68:/etc/openvpn$ sudo sysctl -w net.ipv4.ip_forward=1
net.ipv4.ip_forward = 1
ubuntu@ip-172-31-35-68:/etc/openvpn$ sudo echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf
-bash: /etc/sysctl.conf: Permission denied
ubuntu@ip-172-31-35-68:/etc/openvpn$ sudo su
root@ip-172-31-35-68:/etc/openvpn# sudo echo "net.ipv4.ip_forward = 1" >> /etc/sysctl.conf
root@ip-172-31-35-68:/etc/openvpn# exit
exit
ubuntu@ip-172-31-35-68:/etc/openvpn$ sudo sysctl -p
net.ipv4.ip_forward = 1
ubuntu@ip-172-31-35-68:/etc/openvpn$ sudo iptables -t nat -A POSTROUTING -s 172.31.0.0/24 -o ens5 -j MASQUERADE
ubuntu@ip-172-31-35-68:/etc/openvpn$ ip route
default via 172.31.32.1 dev ens5 proto dhcp src 172.31.35.68 metric 100
172.31.0.2 via 172.31.32.1 dev ens5 proto dhcp src 172.31.35.68 metric 100
172.31.32.0/20 dev ens5 proto kernel scope link src 172.31.35.68 metric 100
172.31.32.1 dev ens5 proto dhcp scope link src 172.31.35.68 metric 100
ubuntu@ip-172-31-35-68:/etc/openvpn$
i-052cc222bcb5a7b9f (ubuntu-server)
PublicIPs: 3.108.41.168 PrivateIPs: 172.31.35.68
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

STEP 16:

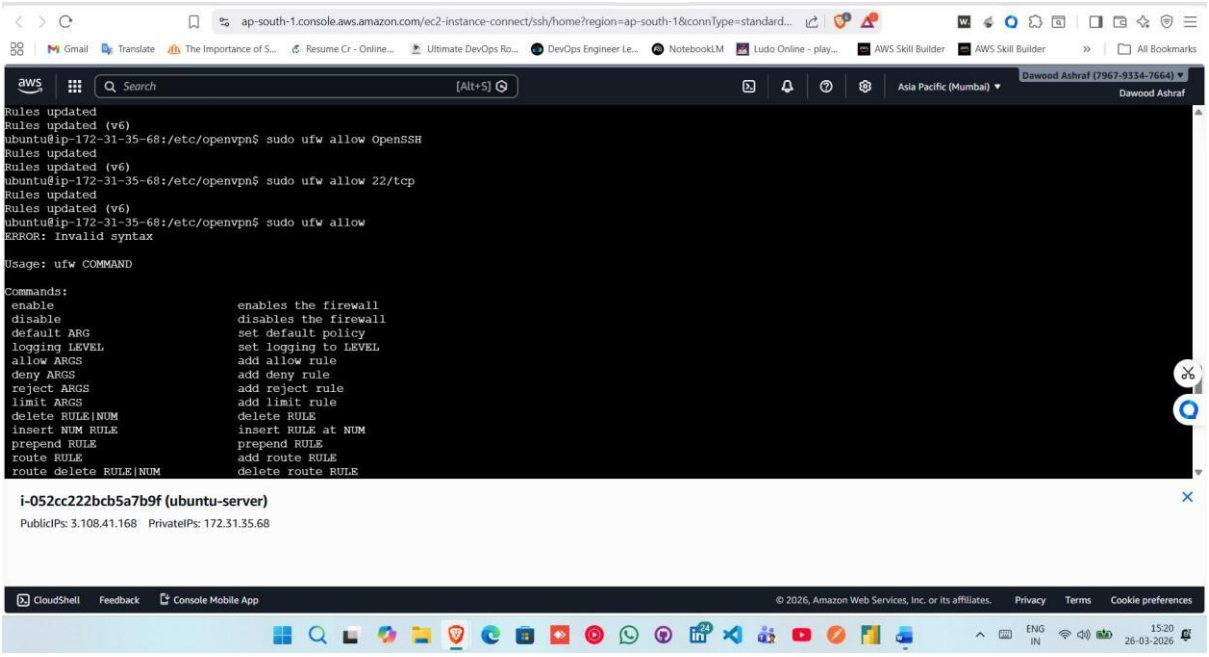
*activating the **Uncomplicated Firewall (UFW)**. This is your EC2 instance's internal security guard. While the AWS Security Group acts as the "Outer Gate," UFW acts as the **"Inner Door."***

Gemini said

*By running these commands, you are activating the **Uncomplicated Firewall (UFW)**. This is your EC2 instance's internal security guard. While the AWS Security Group acts as the "Outer Gate," UFW acts as the **"Inner Door."***

The Breakdown of Your Security Rules

1. ***sudo ufw allow 1111/udp:***
   - *This opens the specific "hole" in your firewall for your **OpenVPN traffic**.*
   - *Since you configured your server.conf to use port **1111** and protocol **UDP**, this is mandatory. Without it, you'll be able to "see" the server from your laptop, but the server will refuse to talk back.*
2. ***sudo ufw allow OpenSSH & sudo ufw allow 22/tcp:***
   - ***Crucial Step:** These two commands do the same thing (allowing SSH access).*
   - *By allowing port 22, you are ensuring that you don't **lock yourself out** of your own server. If you enable UFW without allowing SSH, your current connection will stay alive, but the moment you log out, you will never be able to log back in.*
3. ***sudo ufw enable:***
   - *This "flips the switch." The firewall is now active and enforcing the rules you just wrote.*



STEP 17:

*Go into **/etc/ufw/before.rules** to handle Network Address Translation (**NAT**). While previous **iptables command** was a "quick fix," editing this file makes that fix permanent and professional.*

By editing this file, you are telling the firewall: "Every time you start up, before you do anything else, set up the routing rules that allow my VPN traffic to reach the internet."

*Command:<./ **sudo nano /etc/ufw/before.rules>***

*Type the below before the **filter** in the before.rules*

 # NAT table rules

****nat***

:POSTROUTING ACCEPT [0:0]

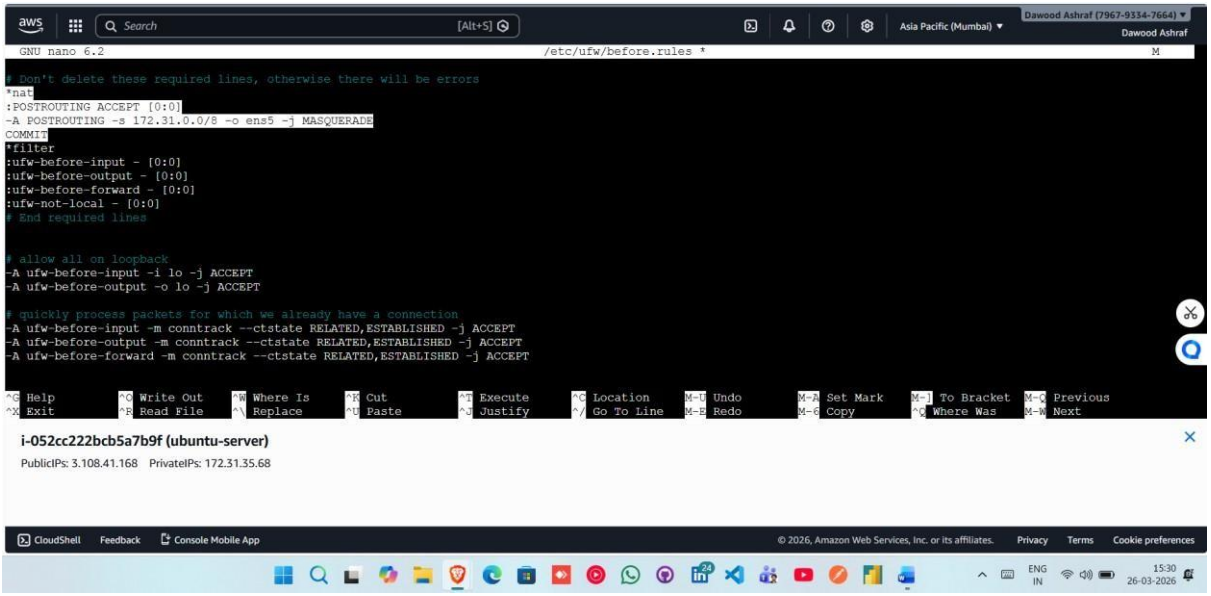
 # Forward traffic from the VPN client through the internet interface

-A POSTROUTING -s 172.31.0.0/24 -o ens5 -j MASQUERADE

COMMIT

The Breakdown of those rules:

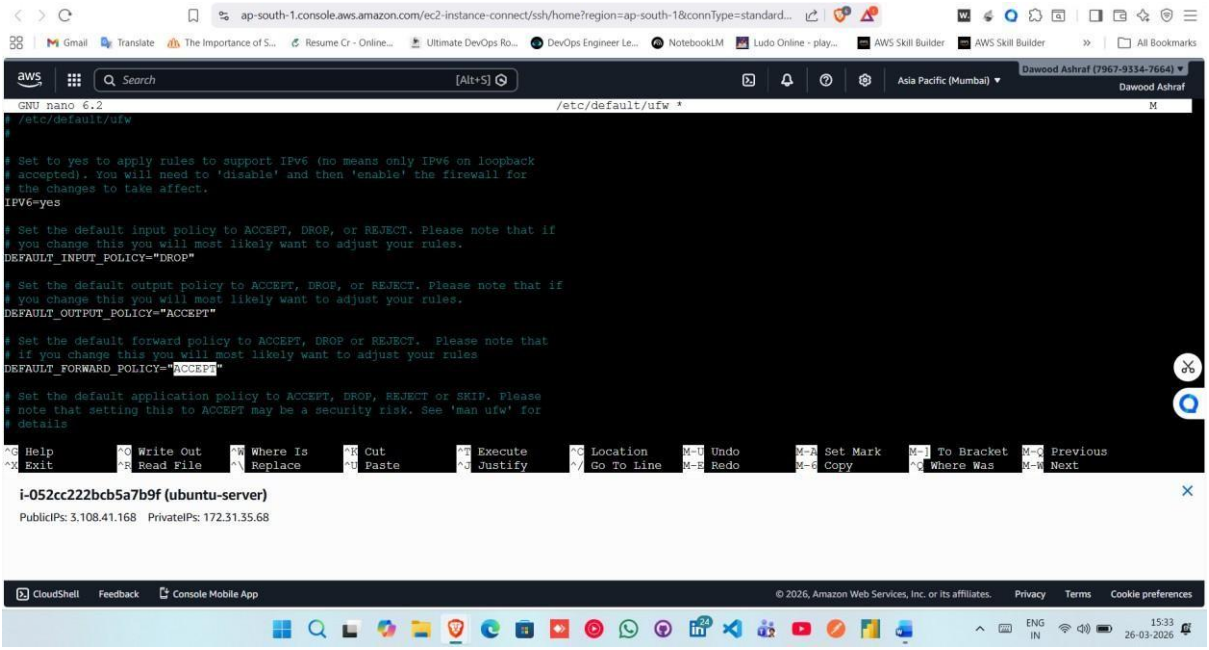
- ****nat:** This tells UFW to switch from "filtering" (blocking/allowing) to "address translation" (routing).*
- ***POSTROUTING ACCEPT:** This initializes the routing chain.*
- ***-A POSTROUTING ... -j MASQUERADE:** This is the exact same logic as your iptables command. It hides your laptop's private VPN IP behind the EC2 instance's Public IP so you can browse the web.*
- ***COMMIT**: This is the "Save" button for that specific block of code.*



STEP 18:

Change the default-forward policy inside the /etc/default/ufw to ACCEPT

*Command: <**sudo nano /etc/default/ufw**>*



STEP 19:

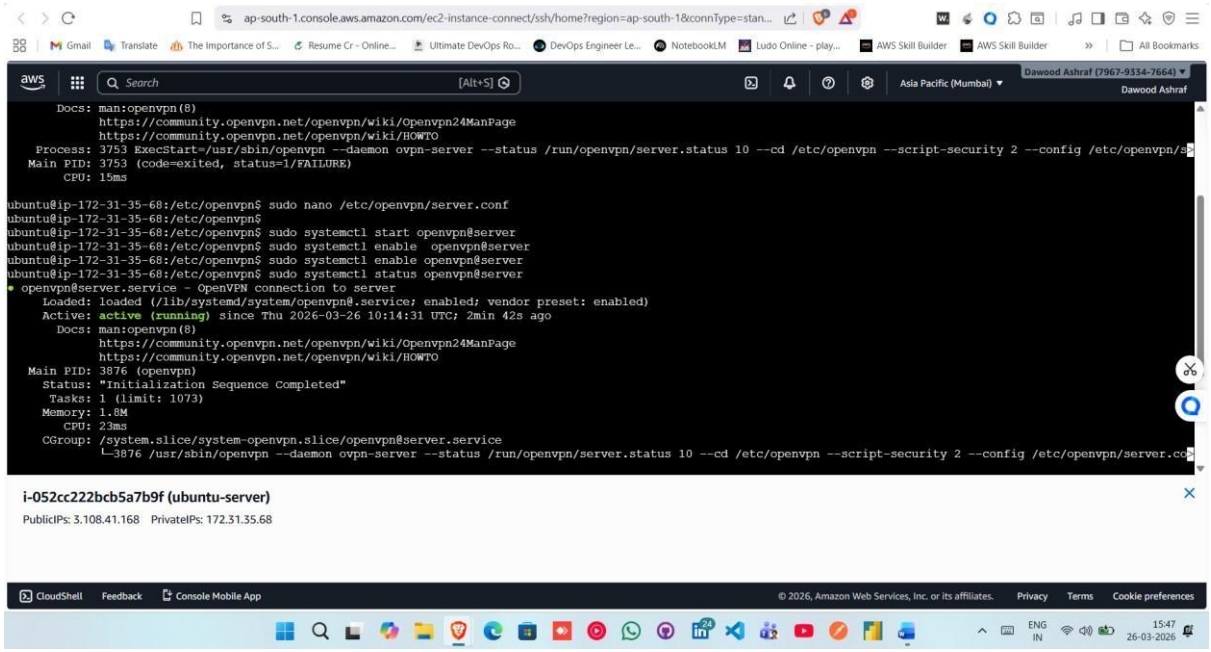
Enable and start the vpn service:

*Command: <**sudo systemctl start openvpn@server**>*

- ***The Action**: This starts the VPN right now.*
- *The Magic **@server**: This is a "template" feature. It tells OpenVPN to look for a specific configuration file named **/etc/openvpn/server.conf.** If you had a second VPN file named **work.conf**, you would run start **openvpn@work.***
- *The Result: Your EC2 instance opens port 1111, loads your certificates, and begins listening for your laptop to connect.*

*Then command: <**sudo systemctl enable openvpn@server**>*

- *The Action: This sets the VPN to Auto-Start.*
- *The Result: If your AWS instance reboots (due to a power flicker or an update), the VPN will automatically turn itself back on without you having to log in and type the start command again. This is essential for a "production-ready" cloud project.*

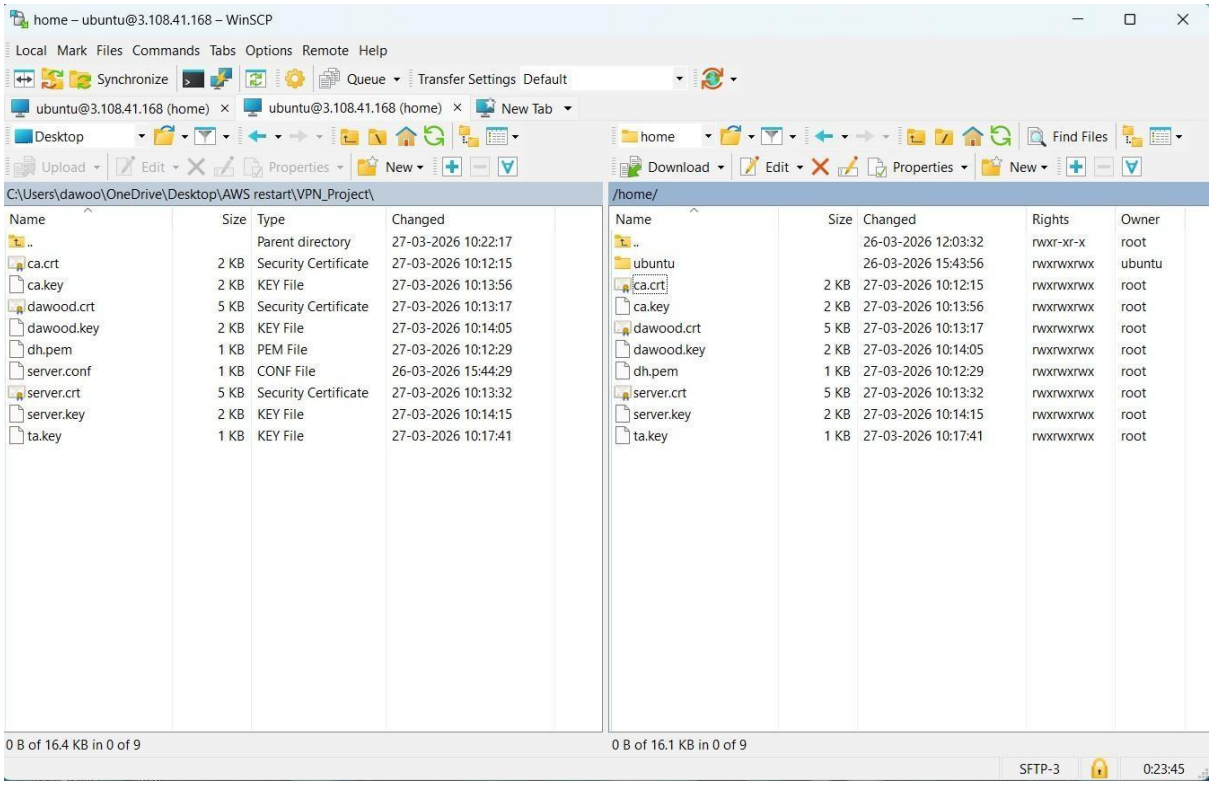


STEP 18:

Copying the required files inside your local computer to access the vpn that is currently running.

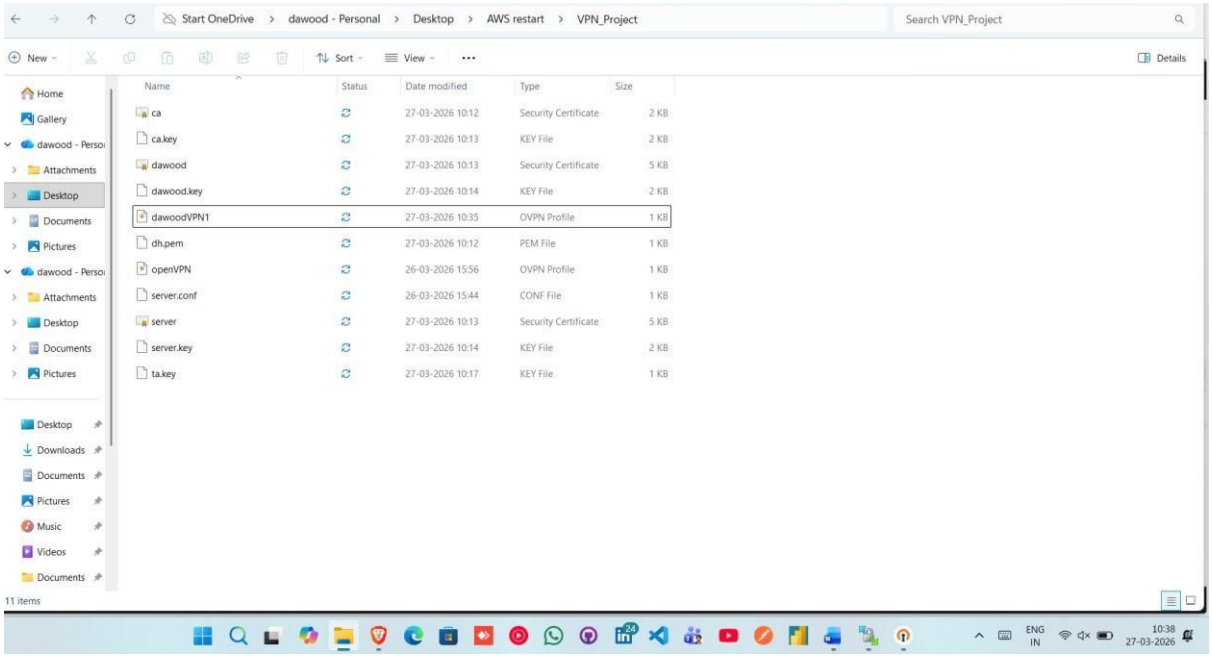
*Using **WinSCP.***

Make sure the permissions are changed for the required files so that they can be easily transferred.



STEP 21:

*Creating a .**opvn** file with all the required details that we need to import to the openvpn software to make the connection.*

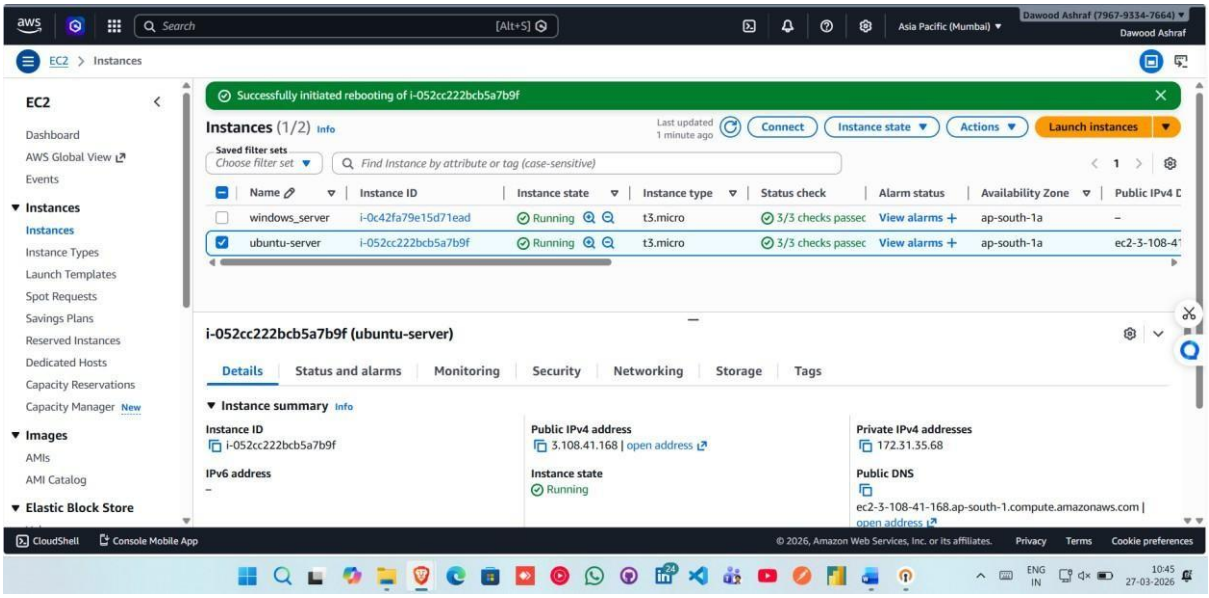


Importing the client.ovpn/dawoodVPN1.ovpn file to openvpn and making the connection to check if working.

STEP 22:

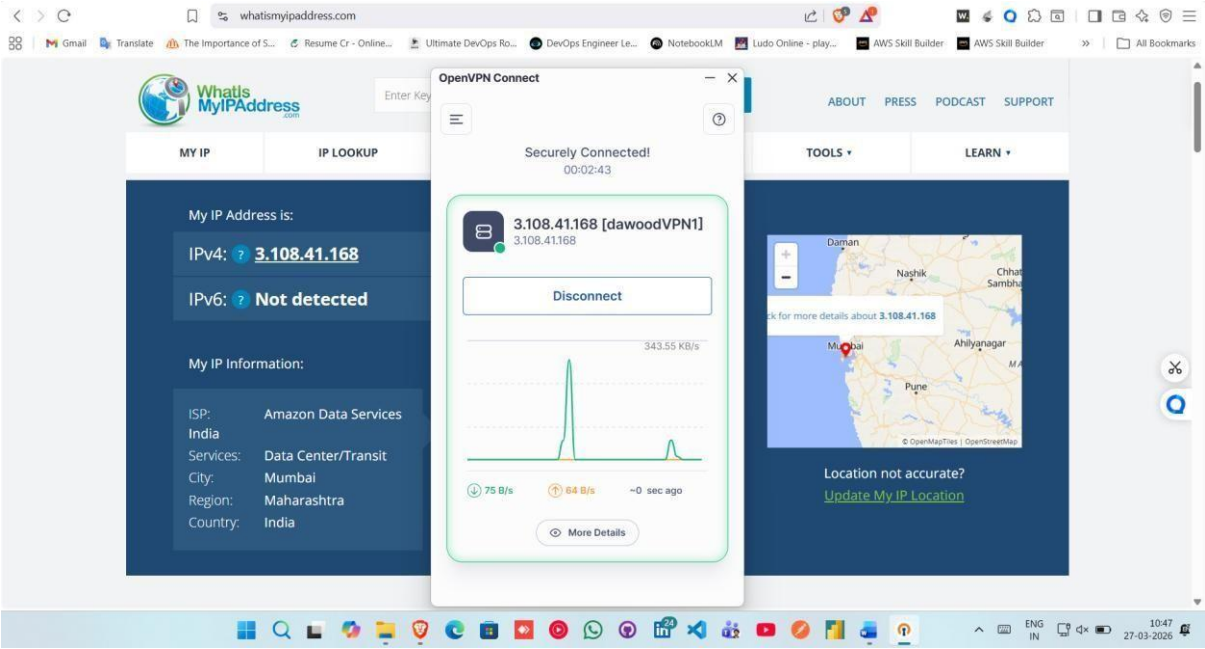
Rebooting the ubuntu instance for the internet to work properly.

STEP 23:



STEP 22:

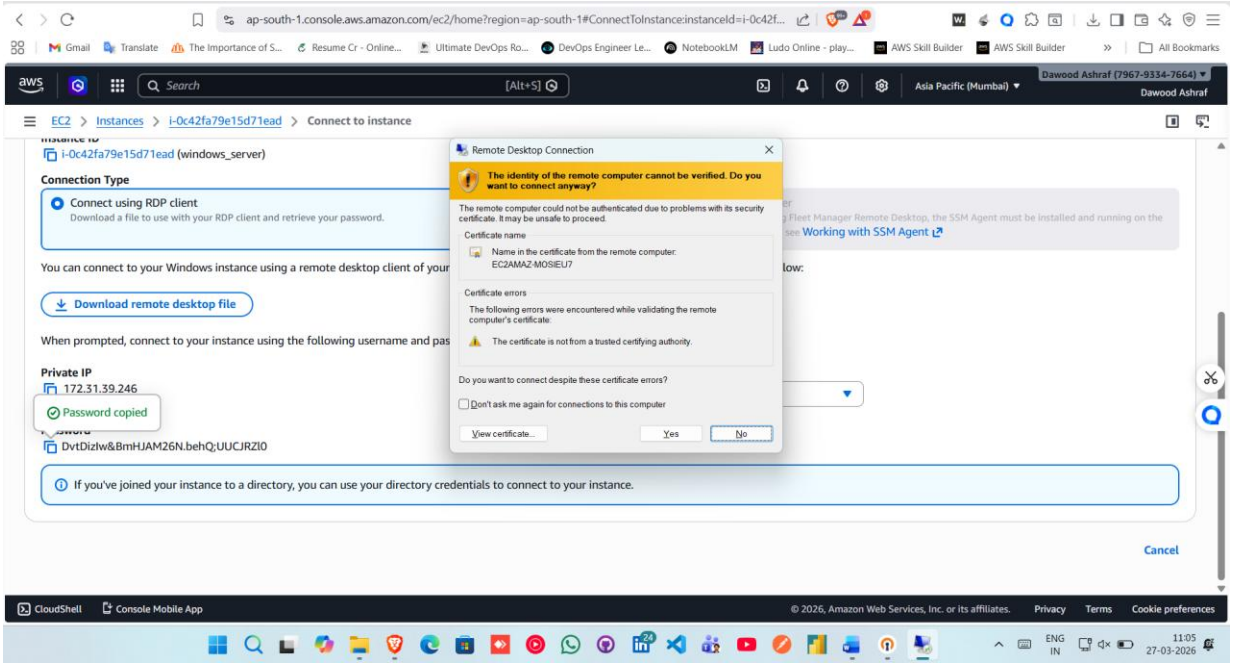
Reconnecting inside the openvpn software after rebooting and connection successful.

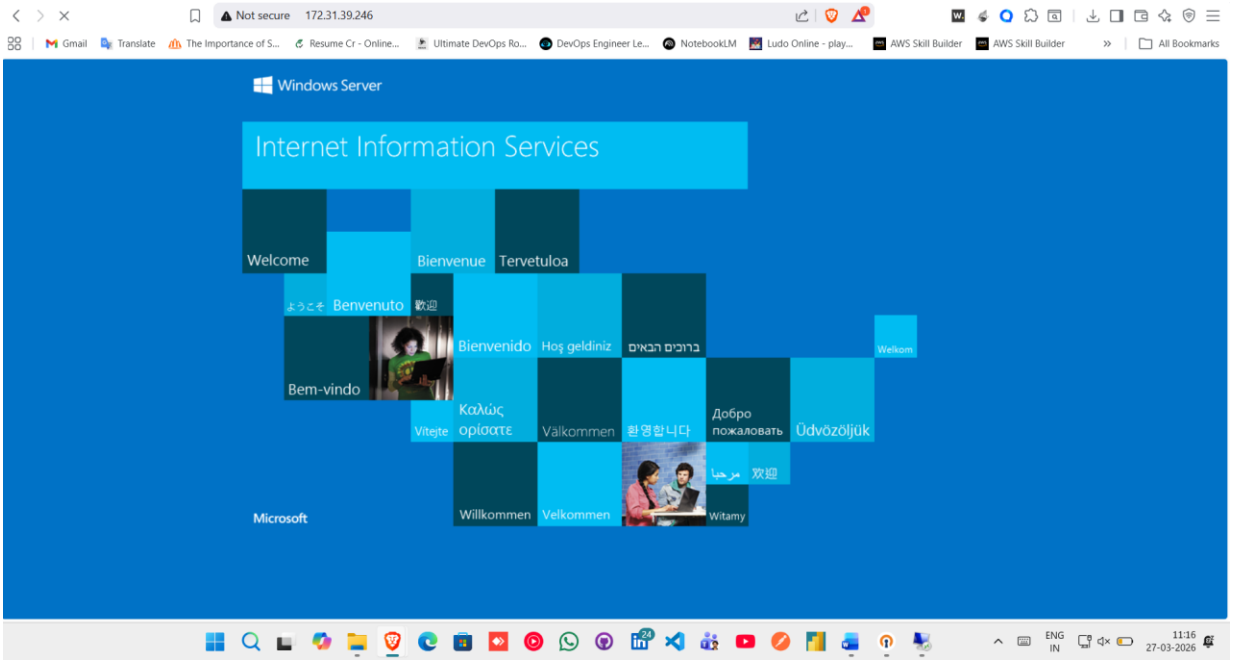


STEP 24:

STEP 23:

Trying to connect using this VPN network to a private windows instance and trying to host a website on it.



STEP 24:

Hosted a website on a private instance and accessed it via owned VPN.